```
int main(void)
{
    int **p; /* declaração de uma matriz com duas dimensões */
    int i,k;

    aloca(&p,4,5); /* passando para a função o endereço de memória do ponteiro */

    for( i=0;i<4;i++)
    {
        for( k=0;k<5;k++)
        {
            p[i][k] = i + k;
        }
    }

    for( i=0;i<4;i++)
    {
        for( k=0;k<5;k++)
        {
            printf("%d  ", p[i][k]); /* referência aos elementos através de linha e coluna */
        }
        printf("\n");
        }
    return 0;
    }
```

2.7.1 Controle de agenda com ponteiros de estruturas e vetores

O programa 2.9 contém várias formas de manipulação e alocação de ponteiros com vetores. O objetivo do programa é criar um cadastro simples de agenda (em memória) com inclusão de novas entradas e alteração, consulta, exclusão e pesquisa (ordenada) de entradas já existentes.

Programa 2.9: Exemplo completo de uso de vetor (ponteiros) de estruturas

```
/* agenda.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* função getch */
#ifdef DOS
#include <conio.h>
#else
```

```
#include <curses.h>
#endif

typedef struct agenda
{
    char nome[40];
    char email[40];
    int telefone;
} AGENDA;

void aloca( AGENDA **pAgenda, int *piEntradas );
void consulta( AGENDA *pAgenda, int iEntradas);

void qs_ordena(AGENDA pAgenda[], int left, int right );
void ordena( AGENDA pAgenda[], int iEntradas );

void excluir(AGENDA **pAgenda, int *piEntradas);
void pesquisar(AGENDA *pAgenda, int iEntradas);
void alterar(AGENDA *pAgenda, int iEntradas);

int main(void)
{
    AGENDA * pAgenda;

    int iEntradas, op;
    iEntradas=0;

    pAgenda = (AGENDA *) malloc(sizeof(AGENDA)); /* alocando espaço para a posição 0
                                                    do vetor */
    if( pAgenda == NULL )
    {
        printf("\nErro de alocacao de memoria.");
        exit(1);
    }

    do
    {
        fflush(stdin);
        printf("\n1 - Inclusao");
        printf("\n2 - Alteracao");
        printf("\n3 - Consulta");
        printf("\n4 - Excluir");
        printf("\n5 - Pesquisar");
        printf("\n9 - Sair");
        printf("\nEntre com uma opcao -> ");
        scanf("%d", &op);
```

```
        if( op == 1 )
        {
        /* farei aqui para ilustrar algumas formas de manipular ponteiros */
        fflush(stdin);

        /* alocação de ponteiros em funções requer trabalhar com ponteiros
              para ponteiros */
        aloca(&pAgenda, &iEntradas);

        printf("*** Inclusao ***");
        printf("\nEntre com o Nome:");
        /* forma 1 - endereço ponteiro inicial + x posições na memória
                    quando se trabalhar com o endereço, deve-se usar -> */
        gets((pAgenda+iEntradas)->nome);
        fflush(stdin);

        printf("Entre com o email:");
        /* forma 2 - endereço ponteiro inicial + x posições na memória
                    quando se trabalhar com ponteiro (conteúdo do endereço ou *),
                    deve-se usar o . (ponto) */
        gets((*(pAgenda+iEntradas)).email);
        fflush(stdin);

        printf("Entre com o telefone:");
        /* forma 3 - trabalhando como vetor */
        scanf("%d", &pAgenda[iEntradas].telefone);
        fflush(stdin);

        iEntradas++;
    }
    else if( op == 2)
    {
        alterar(pAgenda, iEntradas);
    }
    else if( op == 3 )
    {
        /* se o vetor de estruturas vai ser somente lido
           não é preciso passar ponteiro para ponteiro */
        ordena(pAgenda, iEntradas);
        consulta(pAgenda, iEntradas);
    }
    else if( op == 4)
    {
    ordena(pAgenda, iEntradas);
    excluir(&pAgenda, &iEntradas);
    }
    else if( op == 5
```

```
        {
105         ordena(pAgenda, iEntradas);
            pesquisar(pAgenda,iEntradas);
        }
    } while(op!=9);
}
110
void consulta(AGENDA *pAgenda, int iEntradas)
{
    int i;
    for(i=0;i<iEntradas;i++)
115 {
        printf("\n\nRegistro %d", i);
        printf("\n\tNome: %s", pAgenda[i].nome );
        printf("\n\tEmails: %s", pAgenda[i].email );
        printf("\n\tTelefone: %d", pAgenda[i].telefone );
120     getch();
    }
}

void alterar(AGENDA *pAgenda, int iEntradas)
125 {
    char op;
    int i=0;
    char nome[40];
    printf("\n\tDigite o Nome:");
130 fflush(stdin);
    gets(nome);

    for(i=0; i < iEntradas && strncmp( pAgenda[i].nome, nome, strlen(nome))!=0;i++);
    if( i>= iEntradas )
135 {
        printf("\nRegistro nao encontrado");
    }
    else
    {
140     printf("\n\tRegistro %d", i);
        printf("\n\tNome : %s", pAgenda[i].nome );
        printf("\n\tEmail : %s", pAgenda[i].email );
        printf("\n\tFone : %d", pAgenda[i].telefone );
        printf("\n\tConfirma a alteracao ?");
145     op = getch();
        if( op == 'S' || op == 's' )
        {
            fflush(stdin);
            printf("\nEntre com o Nome:");
150         /* forma 1 - endereço ponteiro inicial + x posições na memória
                        quando se trabalhar com o endereço, deve-se usar -> */
```

```
            gets((pAgenda+i)->nome);
            fflush(stdin);

            printf("Entre com o email:");
            /* forma 2 - endereço ponteiro inicial + x posições na memória
                        quando se trabalhar com ponteiro (conteúdo do endereço ou *),
                        deve-se usar o . (ponto) */
            gets((*(pAgenda+i)).email);
            fflush(stdin);

            printf("Entre com o telefone:");
            /* forma 3 - trabalhando como vetor */
            scanf("%d", &pAgenda[i].telefone);
            fflush(stdin);
        }
    }
}

void excluir(AGENDA **pAgenda, int *piEntradas)
{
    char op;
    int i=0;
    char nome[40];
    printf("\n\tDigite o Nome:");
    fflush(stdin);
    gets(nome);
    /* Uso a sintaxe (*pAgenda)[i].nome pelo fato de ser ponteiro de ponteiro.
       Os parênteses neste caso servem para "fixar" a primeira posição da memória, pois
       a linguagem C tende a trabalhar com ponteiros de ponteiros como se fossem
       matrizes (que na prática são ponteiros para ponteiros) */
    for(i=0; i < *piEntradas && strncmp( (*pAgenda)[i].nome, nome, strlen(nome))!=0;i++);
    if( i>= *piEntradas )
    {
        printf("\nRegistro não encontrado");
    }
    else
    {
        fflush(stdin);
        printf("\n\tRegistro %d", i);
        printf("\n\tNome : %s", (*pAgenda)[i].nome );
        printf("\n\tEmail : %s", (*pAgenda)[i].email );
        printf("\n\tFone : %d", (*pAgenda)[i].telefone );
        printf("\n\tConfirma a exclusao ?");
        op = getch();
        if( op == 'S' || op == 's' )
        {
            /* copio o último elemento para o elemento corrente */
            (*pAgenda)[i] = (*pAgenda)[(*piEntradas)-1];
```

```
            (*piEntradas)--;
            /* excluo o último elemento com realoc */
            aloca(pAgenda, piEntradas);
        }
    }
}

void aloca( AGENDA **pAgenda, int *piEntradas )
{
    (*pAgenda) = (AGENDA *)(realloc(*pAgenda,
                (*piEntradas+1)*sizeof(AGENDA)));
    if( *pAgenda == NULL )
    {
        printf("\nErro de re-alocacao de memoria");
        exit(1);
    }
}

void pesquisar(AGENDA *pAgenda, int iEntradas)
{
    char op;
    int i=0;
    char nome[40];
    printf("\n\tDigite o Nome:");
    fflush(stdin);
    gets(nome);

    for(i=0; i < iEntradas && strncmp( pAgenda[i].nome, nome, strlen(nome))!=0;i++);
    if( i>= iEntradas )
    {
        printf("\nRegistro nao encontrado");
    }
    else
    {
        do
        {
            fflush(stdin);
            printf("\n\n\tRegistro %d", i);
            printf("\n\tNome : %s", pAgenda[i].nome );
            printf("\n\tEmail : %s", pAgenda[i].email );
            printf("\n\tFone : %d", pAgenda[i].telefone );
            printf("\n\tProximo ?" );
            op = getch();
            i++;
            if(i>=iEntradas)
            {
                i = 0;
            }
```

```
        } while( op == 'S' || op == 's');
    }
}

void ordena( AGENDA pAgenda[], int iEntradas )
{
    qs_ordena(pAgenda, 0, iEntradas-1 );
}

void qs_ordena(AGENDA pAgenda[], int left, int right )
{
    register int i, j;
    char * x;
    AGENDA t;

    i = left;
    j = right;
    x = pAgenda[(left+right)/2].nome;

    do
    {
        while(strcmp(pAgenda[i].nome,x)<0 && i<right) i++;
        while(strcmp(pAgenda[j].nome,x)>0 && j>left) j --;
        if( i<=j )
        {
            t = pAgenda[i];
            pAgenda[i]=pAgenda[j];
            pAgenda[j]=t;
            i++;
            j--;
        }
    } while( i<=j );
    if( left < j ) qs_ordena(pAgenda, left, i);
    if( i<right) qs_ordena(pAgenda, i, right );
}
```